

UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Denkleiers • Leading Minds • Dikgopolo tša Dihlalefi

COS700 Research Report

Metaheuristic Wrapper-Based Feature Selection for DNS Exfiltration Detection on an ISP-Derived Resolver Dataset

Student number: u20704209

Supervisor(s):
Dr Avinash Singh

Abstract

DNS tunnelling enables covert communication and data exfiltration through DNS queries and responses. This study evaluates whether metaheuristic wrapper-based feature selection can identify compact subsets of 17 engineered DNS features while maintaining detection performance and reducing classifier computational cost. The evaluation uses an ISP-derived DNS exfiltration dataset containing benign and attack records.

Random Forest and XGBoost models using all 17 features provide the reference baselines. Ant Colony Optimisation (ACO), Particle Swarm Optimisation (PSO), and Enhanced Reinforcement Learning-Guided Grey Wolf Optimisation (Enhanced RLGWO) search for subsets containing between eight and ten features. Candidate subsets are fitted using a Random Forest evaluator on 500,000 training records and scored on 150,000 validation records using weighted F1-score, false-positive rate, and a penalty for deviation from nine selected features. Each optimiser's selected subset is subsequently evaluated using both final classifiers.

Enhanced RLGWO achieved the highest attack-class F1-score among the three feature-selection methods for both classifiers. With XGBoost, it increased F1-score from 0.997016 for the full-feature baseline to 0.997765 and reduced total classification errors from 259 to 194. Recorded XGBoost training and batch inference times decreased by 13.36% and 6.93%, respectively. However, the full-feature Random Forest remained the strongest detector overall, and PSO achieved the largest Random Forest training-time reduction.

The results show that the best feature-selection method depended on the classifier used; no method performed best in every test. Repeated classifier evaluations with three random seeds produced similar attack-class F1-scores when the selected feature subsets were kept fixed. This study measured classifier training and batch prediction time only. It did not measure feature-extraction cost, metaheuristic search time, or end-to-end deployment latency.

Keywords:

DNS tunnelling detection, DNS exfiltration, Metaheuristic feature selection, Particle Swarm Optimisation (PSO), Enhanced Reinforcement-Learning-Guided Grey Wolf Optimiser (Enhanced RLGWO), Ant Colony Optimisation (ACO), Random Forest, XGBoost, Intrusion detection, ISP DNS traffic

1 Introduction

The Domain Name System (DNS) is a protocol described as the Internet's phonebook. It translates human-readable domain names into numerical IP addresses [1], [2] . DNS is an essential component of the internet's infrastructure; hence it is often permitted through security controls and policies, which can make it attractive to attackers for misuse [3]. As a result, the protocol can be exploited in attacks such as DNS spoofing, DNS hijacking, and DNS tunnelling. [1]. DNS tunnelling is a mechanism that enables covert data transfer between two systems by encapsulating data within DNS queries and responses[1], [2]. According to Abualghanam et al. [4], there has been notable misuse in DNS tunnelling concerning data breaching [4], creating urgency for researchers to address this problem. In response, various approaches have been proposed. These include traditional signature-based methods such as fingerprinting (identifying unique patterns), payload analysis (examining the contents of DNS messages), traffic analysis (studying data flow), and statistical features [3], [5].

More recent work has introduced machine learning approaches, including supervised classification and unsupervised anomaly detection [2], [4]. Traditional techniques often assume predictable behaviour from attackers; however, attackers adapt known malicious indicators over time to mimic normal DNS traffic [1]. Therefore, techniques relying on fingerprints, payload analysis, or minor statistical indicators can be ineffective for detection. Machine learning can assist with this dilemma by utilizing multiple DNS features, therefore improving the effectiveness of intrusion detection systems without compromising network performance [2].

Over the past decade, comparative research on machine learning-based intrusion detection methods has been systematically replicated and tested [5]. These methods are grouped into three categories: traditional machine learning, ensemble learning, and deep learning [5].

Zhang [5] concluded that ensemble learning methods, such as XGBoost and Random Forest , perform well in intrusion detection compared to other categories. However, several limitations remain. First, many existing datasets do not reflect real-world traffic conditions [5], [2] ,[4], [3], raising concerns about the validity of the assessed model's performance. Second, most proposed approaches are based on manual preprocessing and feature design and selection [1], [4]. Finally, proposed models/pipelines usually exhibit significantly higher computational overhead [3], [4].

This study addresses the limitations of previous research by proposing a machine-learning-based method to detect DNS tunnelling. The first objective is to evaluate model performance using Žiža et al. [1] publicly available dataset [1] , which was derived from a internet service provider (ISP) DNS resolver and reflects more realistic benign and malicious DNS activity than the private, controlled, and favourable settings commonly used in earlier work. The second objective is to compare metaheuristic optimisation algorithms, namely EnhancedRLGWO, PSO, and ACO, as automated feature selection for dimensionality reduction, with the goal of eliminating manual preprocessing and feature selection/engineering.

[1] DNS Exfiltration Dataset [6], available on Mendeley Data.

In addition, motivated by Žiža et al ., this study will consider both single-request and aggregate DNS features, as their findings suggest that the joint use of these feature categories provides more reliable detection than using them in isolation, especially under adversarial DNS behaviour. This study investigates whether machine learning,using flow-level DNS features, can improve the detection of covert DNS tunnelling activity under more realistic traffic conditions. This investigation will further evaluate automated feature-selection methods to reduce the dependence on manual preprocessing and feature design.

2 Problem Statement

As observed in early studies, common models are evaluated on the basis of controlled or synthetic datasets, making proposed findings unreliable should the solution be tested under realistic DNS traffic conditions. Žiža et al. [1] addresses the challenge by providing a dataset extracted from an operational ISP resolver. In addition, Žiža et al. [1] evaluated two known models, namely Random Forest and XGBoost-using the dataset, to further prove the high detection performance the models can produce using the 17 manually engineered chosen DNS features. The DNS features chosen include both the single-request and aggregate categories.

The approach Žiža et al. [1] used still relies on a rigid, manually designed feature set and does not test whether all features are necessary for more robust detection. Žiža et al [1] stated that one of the weaknesses in their study is mainly caused by the single request features that dominate the real-time budget which impacts practical deployability, should the pipeline proposed be deployed on real DNS infrastructure. Therefore, an investigation will be done to test whether automated feature selection methods can identify effective/useful features that can (a) reduce computational cost (b) conserve a good detection performance even under more realistic adversarial DNS tunneling behaviour.

This investigation will be addressing the identified gap by utilizing chosen metaheuristic feature-selection algorithms to Žiža et al public dataset to look for compact/effective subsets of the 17 DNS features. The following research question that will be addressed is:

1. Can metaheuristic feature-selection methods find reduced subsets of the 17 (9 single-request and 8 aggregate) DNS features that maintain accurate and robust DNS tunneling detection on Žiža et al. 's realistic ISP DNS dataset?
    (a) What detection performance (precision, recall, F1-score) do standard ensemble models (Random Forest, XGBoost) achieve on Žiža et al. 's dataset using all 17 manually engineered DNS features?
    (b) How do different metaheuristic feature-selection algorithms (EnhancedRLGWO, PSO, ACO) compare in terms of the size of the selected feature subset on the same dataset?
    (c) How do compact feature subsets affect recorded classifier training time and batch prediction time compared with using all 17 features?

3 Literature Study

DNS tunnelling is recognised as a covert communication channel. It enables command-and-control operations and data exfiltration by encapsulating payloads in DNS queries and responses. These tunnels can be attractive to malicious actors because DNS traffic is frequently allowed through firewalls and rarely inspected deeply. Previous detection methods relied on signature-based approaches, such as fingerprinting, payload analysis, and traffic analysis. These methods commonly used simple statistical features, such as query length, character entropy, and query frequency. They commonly use domain name character features to distinguish benign from malicious DNS traffic. Aiello et al. [7] showed that tunnels can be detected using aggregation monitoring which involves examining groups of DNS packets over time and combining them with supervised machine learning on statistical traffic profiles.As a result, computationally expensive deep packet inspection can be avoided. However, they highlighted a challenge that silent intruders aim to cause only small statistical deviations enough to avoid triggering fixed thresholds causing missed detection in realistic low and slow scenarios [7].

Building on a feature-based viewpoint, [8] proposed a three-stage FTPB DNS detection method. This method combined character level feature extraction with Term Frequency–Inverse Document Frequency(TF-IDF), Principal Component Analysis(PCA), and Bootstrap Aggregating (Bagging) algorithms. Their pipeline filters noticeable tunnel domains using TF-IDF text features, then PCA is used for dimensionality reduction, then finally a bagged ensemble classifier is applied to detect tunnels. They reported high detection and low false positive results, illustrating how sophisticated feature engineering can improve detection over rule-based or threshold-dependent systems.

Many studies have applied machine learning to network intrusion detection systems (NIDS) -reporting high accuracies which are often above 95% or even close to 99% on benchmark datasets [5] [9]. These include traditional classifiers such as decision trees , Support vector machines, and ensembles (Random Forest, Gradient Boosting, XGBoost) and more recently introduced deep learning architectures (CNNs, RNNs, transformers). Comparative work by Zhang et al.[5] on network intrusion detection methods, confirms that ensemble models are more effective, or rather outperform traditional classifiers and often match or outperform deep learning approaches on NIDS benchmarks-especially when training data is limited. However, a recurring limitation was identified that many of these evaluations rely on synthetic, lab-generated, datasets where malicious traffic is clearly separable from benign traffic Žiža et al.[1] .

Žiža et al. raised this concern explicitly for DNS exfiltration: They argued that reports that come with high accuracies came from experiments in idealised environments where DNS tunnels requests have distinguishable features which might not hold and may not be preserved in real-world deployments . Abualghanam et al.[4] extended this work by developing a real-time machine learning system for DNS exfiltration that uses a compact set of traffic features. They demonstrated that a reduced feature set has the ability to still achieve high detection performance while supporting low-latency deployment. This suggests that having more features does not necessarily equate to better accuracy, however a good set of features can produce better accuracy at a lower computational

cost.

To address challenges of model evaluation credibility and realistic data scarcity, Žiža et al [1] introduced a publicly available dataset [2]. The dataset was collected from an operational ISP resolver containing about 51 million DNS requests. The dataset has both regular traffic and exfiltration generated by tools such as DNSExfiltrator and Iodine, as well as additional modified exfiltrations with randomised waiting times and reduced entropy to mimic stealthy adversarial behaviour. Žiža et al. [1] manually defined 17 DNS features. These included nine single-request lexical features(domain length, entropy, and English-word statistics). Eight aggregate features describing temporal and statistical behaviour over sequences of requests. [1] Žiža et al.'s work showed that ensemble models such as Random Forests and eXtreme Gradient Boosting (XGBoost) can reliably detect DNS exfiltration, reporting F1-scores close to 1.0 on their DNSExfiltrator dataset under varied parameter settings. After enriching the training data with their modified exfiltrator, models trained on both single-request and aggregate features (TD-40) or aggregate features alone (`AF-40`) again reached precision ≈ 1.0 and recall around 99% for modified exfiltration, while single-request–only models (SR-40) detected less than one-third of such traffic. However , they also observed that single-request features are computationally expensive on resolvers, raising concerns about scalability. To elaborate further, this is due to the requirement of having to preprocess every query string while also having to search through about 60 000 common english words. They noted through their performance analysis that feature computation processed 4,000 DNS requests per second while the trained XGBoost model can classify about 1.5 million examples per second . This ultimately means that feature extraction is the biggest contributor to computational cost raising concerns on scalability on more realistic ISP environments.

To further support the model choice made by Žiža et al. , Zhang et al.[5] confirms that ensemble models are more effective, or rather outperform traditional classifiers and deep learning approaches on NIDS benchmarks, thereby strengthening the choice of using Random Forest and XGBoost as baselines for DNS-tunnelling detection on realistic datasets. Sui et al.[3] noted persistent limitations across protocol layers;there is a heavy reliance on synthetic or lab-generated datasets, manually crafted feature engineering, and computational overhead, all of which motivate efforts to produce compact, efficient feature subsets from realistic DNS data rather than on complex models and features.

Recent research increasingly applies optimisation and metaheuristic techniques to DNS tunnelling and encrypted DNS. Sammour et al. [2] compared supervised machine learning methods(SVM, Naive Bayes, J48) for DNS tunnelling. They later proposed a hybrid reinforcement-learning and metaheuristic framework to improve detection performance . Their hybrid system improved over their supervised baselines, this work demonstrated that search-based optimisation can refine detection pipelines. Talabani et al[10] focused on DNS-over-HTTPS (DoH) tunnelling. They used Ant Colony Optimisation to automate feature selection. As a result, metaheuristic selection from the CIRA-CIC-DoHBrw-2020 dataset [11] achieved near-perfect detection accuracy. It also reduced feature dimensionality and computational cost.

[2]DNS Exfiltration Dataset [6], available on Mendeley Data.

Mitsuhashi et al. [12] further extended DNS tunnelling research to encrypted DNS (DNS-over-HTTPS). They successfully recognised malicious DNS tunnel tools by learning patterns from DoH flows. They built a three stage classifier using gradient boosting decision libraries trained on CIRA-CIC-DoHBrw-2020 dataset and their private DoH-Tunnel-Traffic-HKD dataset and observed that their classification accuracy was 98% (F1=97.6%) at Stage 3 ,for being able to distinguish six tunnel tools. They also managed to maintain Stage-1 and Stage-2 accuracies above 99.8% for DoH filtering and benign-versus-malicious DoH classification. Talabani's and Mitsuhashi's both highlight that one, tunnel behaviour and DoH traffic have learnable patterns even when payloads are encrypted; second optimisation has the potential to significantly improve detection robustness while remaining compatible with real time deployment constraints .

Furthermore, Jerabek et al. [13] proved that DoH traffic can be differentiated from generic HTTPS traffic using simple standard flow records (NetFlow/IPFIX) and machine learning models. They also emphasised that most detection systems rely on specialised, additional monitoring tools or complex features, which often make deployment and operation in real networks difficult. Further suggesting that DNS tunnels exhibit learnable traffic patterns enough that they can be used to support scalable, low-overhead feature sets instead of relying solely on deep learning architectures.

Sharma et al. [14] focused on lightweight, zero-day-capable DNS tunnel detection. They propose a three-layer architecture where, in the first layer, they used OptiTuneD- a framework for filtering legitimate domain names , the second used a BERT-based classifier to distinguish legitimate domains from suspicious domains , and the third layer separated zero-day tunnels from rare legitimate queries using a DNS probing technique. This approach achieved high recall about 98.7% , precision of about 97.5%, and F1-scores around 97.95% including an average processing time of 473.25 ms per DNS packet(across the three layers) on three public datasets. The layered approach demonstrates that accuracy and efficiency can be balanced for both known and unknown tunnels. However, it relies on specialised components, such as a large legitimate-domain list, a deep language model, and an active-probing stage. This increases implementation complexity and deployment overhead compared to flow-level models.

Jerabek's and Sharma's work highlight that complex architectures (layered) with deep models can yield fairly good results (accuracy and zero-day) but they might be difficult to integrate in high-throughput environments an perhaps a simpler approach can be more deployment friendly.

Recent intrusion-detection studies show that metaheuristics and hybrid approaches can improve feature selection and boost classifier performance. Li et al. [15] used feature filtering and PCA with gradient-boosting ensemble to improve semi-supervised NIDS performance, highlighting that dimensionality reduction can increase accuracy and reduce training time. Awade et al. [16] applied hybrid metaheuristic optimisation (Particle Swarm Optimisation (PSO) and Chaotic Butterfly Optimisation Algorithm (CBOA)) to optimise feature subsets and parameters for an LTSM detector, reporting increased detection rates and reduced false positives compared to the untuned baselines. Mazini et al. [9]propose an anomaly network-based IDS that combines Artificial Bee Colony (ABC) for feature selection with AdaBoost as the classifier, treating the

choice of features as an optimisation problem. On NSL-KDD and ISCXIDS2012 they report higher detection rates (close to 99%) and reduced false positives than traditional IDS methods, suggesting that swarm-intelligence algorithms can lead to higher accuracy even in high dimensional datasets.[9].

KRTunnel, by Wang et al. [17], targets DNS tunnel detection on Android devices. The method uses manually extracted features from DNS requests and responses and employs the Isolation Forest model. However, the authors note that some features (such as TTL) can be invalidated if attackers modify them without changing tunnelling behaviour. This underscores the brittleness of manually selected features. Liang et al. [18] proposed a novel pipeline: Feature Extraction, CNN and Clustering (FECC), where raw DNS payloads are processed by a CNN feature extractor and evaluates the homogeneity and exclusivity of those features with a clustering-based loss. They observe that payloads crafted to mimic legitimate DNS queries are difficult to distinguish, and that the method's high memory usage can hinder real-time deployment. This review demonstrates that machine learning and metaheuristics can be effective for network security detection and suggests that similar strategies could be promising for detecting DNS tunnelling.

4 Methodology

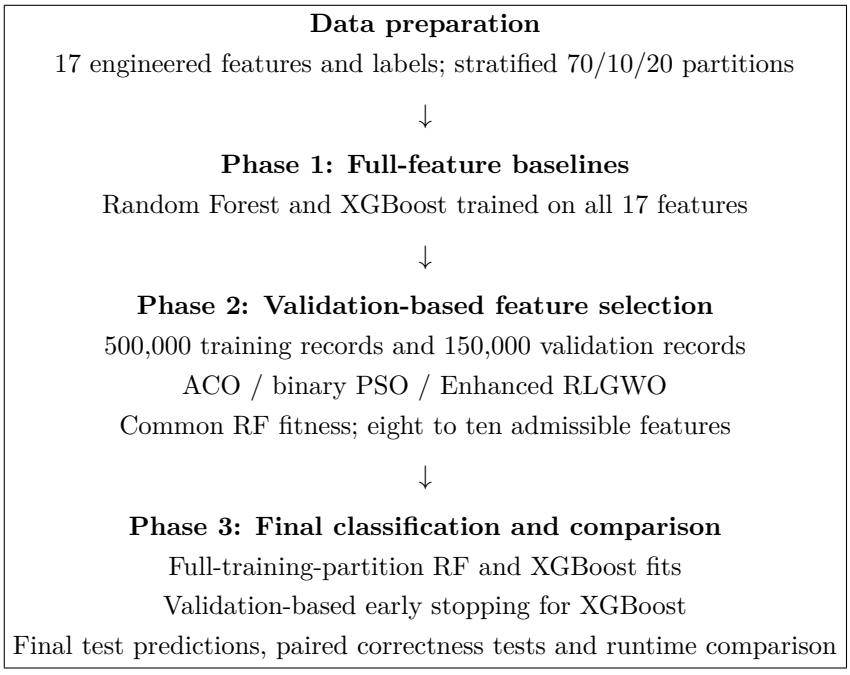


Figure 1: Implemented three-phase experimental workflow. Candidate fitness uses Random Forest only; both classifiers evaluate the selected subsets.

4.1 Research design

The study used three experimental phases. First, full-feature Random Forest and XGBoost models established reference baselines. Second, ACO, binary PSO and Enhanced RLGWO selected subsets from the same 17-feature pool using a common Random Forest fitness function. Third, final classifiers were trained on the selected features and compared with their corresponding baselines using detection metrics and classifier runtime.

Candidate fitness was evaluated on validation records, not test records. The searches and final model evaluations were rerun, producing the selected subsets and results reported here. The baseline implementation was informed by Žiža et al. rather than treated as an exact reproduction [1].

4.2 Dataset and feature representation

The study used the DNS exfiltration dataset published by Žiža et al. [1, 6]. Benign traffic was derived from an operational ISP recursive resolver; generated exfiltration traffic included DNSExfiltrator and Iodine, with additional modified-exfiltrator examples. The presence of real resolver traffic does not mean that all attacks occurred naturally.

The nine single-request features were `len`, `subdomains_count`, `w_count`, `w_max`, `entropy`, `w_max_ratio`, `w_count_ratio`, `digits_ratio` and `uppercase_ratio`. The eight aggregate features were `time_avg`, `time_stdev`, `size_avg`, `size_stdev`, `throughput`, `unique`, `entropy_avg` and `entropy_stdev`. These describe query content and statistics over related request windows, rather than exclusively standard network-flow telemetry [1].

No new feature combinations were constructed. Class 0 denoted benign records and class 1 denoted exfiltration-labelled records. These labels refer to dataset records, not distinct malicious incidents. The source study also labelled some legitimate uses of DNS data transport as exfiltration [1].

The source dataset contains aggregate statistics calculated over batches of ten requests, with overlap between consecutive windows. The corrected experiment reused cached partitions and did not independently audit separation at the request-window, host or domain-group level. This limits claims about independence and generalisation.

4.2.1 Data partitioning

The proposal specified an 80/10/10 training/validation/test allocation, matching the proportions reported by Žiža et al. [1]. The implementation instead used a stratified 70/10/20 allocation with `random_state=42`. Original and modified records were partitioned separately and combined within their corresponding partitions.

The resulting class distributions are reported in Table 1. The 20% test allocation was an experimental design choice, not a proportion demonstrated to be optimal

for this dataset. Stratification preserved class representation but did not establish independence between related traffic windows. This pooled evaluation also differed from the original study's separate experiments varying the proportion of modified-exfiltrator examples available during training.

Table 1: Class distribution of the training, validation and test partitions.

Split	Benign (0)	Attack (1)	Total
Train	24,429,561	151,956	24,581,517
Validation	3,489,937	21,708	3,511,645
Test	6,979,874	43,416	7,023,290

4.3 Classifier setup

4.3.1 Random Forest

Random Forest was used as a baseline classifier and as the classifier within the feature-subset evaluation procedure. The configuration described here applies to final evaluation on the full training partition; the search-stage classifier is described separately. The final Random Forest models used five trees (`n_estimators=5`), a maximum tree depth of 20 (`max_depth=20`), and one parallel job (`n_jobs=1`). No class weighting was applied (`class_weight=None`), and the random seed was fixed at 42 (`random_state=42`).The training partition contained 24,581,517 records.

A five-tree ensemble was used to constrain computational cost, while the depth bound limited individual tree growth. The ensemble size and absence of class weighting are consistent with Žiža et al.'s reported Random Forest configuration. However, their trees were grown without an explicit depth limit, whereas this study imposed (`max_depth=20`). These settings therefore represent the configuration used in this study rather than an exact reproduction of the original implementation

The same explicitly specified Random Forest hyperparameters were used for the 17-feature baseline and all three reduced feature subsets. This avoided changing settings between feature configurations. The default square-root feature-sampling rule was retained, so the number of candidate features considered at each split depended on the number of input features. The final evaluation configuration is summarised in Table 2.

With all 17 features, Random Forest achieved an attack-class F1-score of 0.9990 on the test set. No sensitivity analysis over ensemble size or tree depth is presented; consequently, the results establish performance under the selected configuration rather than demonstrating that it is optimal. Larger ensembles or alternative depth settings could change both detection performance and computational cost.

4.3.2 XGBoost

XGBoost was used as the second baseline classifier. Its final detection-evaluation configuration matched the principal settings reported by Žiža et al.: a binary logistic objective (`objective="binary:logistic"`), validation classification error as the early-stopping metric (`eval_metric="error"`), a training-instance subsampling ratio of 0.1 per boosting iteration (`subsample=0.1`), and a maximum of 100 boosting rounds (`n_estimators=100`). Training used an early-stopping of 10 consecutive rounds without improvement in validation error (`early_stopping_rounds=10`), monitored on the separate validation partition. No class reweighing was applied (`scale_pos_weight=1`), and no sample-specific weights were supplied during fitting. These choices match the listed settings in the original study. The final evaluation configuration is summarised in Table 3.

Table 2: Random Forest configuration used for the baseline and final evaluation of the selected feature subsets.

Parameter	Value
`n_estimators`	5
`max_depth`	20
`n_jobs`	1
`random_state`	42
`class_weight`	`None`

Table 3: XGBoost configuration used for the baseline and final evaluation of the selected feature subsets.

Parameter	Value
`objective`	`binary:logistic`
`n_estimators`	100
`subsample`	0.1
`scale_pos_weight`	1
`eval_metric`	`error`
`early_stopping_rounds`	10
`n_jobs`	4
`random_state`	42

4.4 Metaheuristic feature selection

4.4.1 Optimisers Used

Three metaheuristic algorithms were used as wrapper-based feature selectors: Ant Colony Optimisation (ACO), binary Particle Swarm Optimisation (PSO), and Enhanced Reinforcement Learning-Guided Grey Wolf Optimisation (Enhanced RLGWO). The wrapper evaluation procedure was adapted from the Random Forest-based framework described by Sammour et al.[2] In this approach, each admissible candidate feature subset was evaluated by training a Random Forest classifier using only the selected features and measuring its predictive performance. This performance was combined with the penalties defined in the fitness function to produce a score that guided the subsequent search for improved subsets.

All three optimisers selected from the same pool of 17 features and used the same Random Forest-based fitness function and feature-count constraints. Each optimiser was configured with a population of 40 candidate solutions and a generation limit of 15. These common settings provided a consistent basis for comparison, although equal population sizes and generation counts do not necessarily imply equal numbers of fitness evaluations or equal computational costs. After feature selection, the selected subsets were evaluated using both Random Forest and XGBoost.

- **Ant Colony Optimisation (ACO):** ACO was implemented through a custom `BinaryACO` class adapted from the pheromone-based feature-selection mechanism described by Talabani et al.[10]. Each artificial ant constructed a candidate subset by probabilistically selecting features according to their pheromone levels and heuristic weights. After the subsets were evaluated, pheromone evaporation reduced the influence of previous choices, while reinforcement increased the attractiveness of features belonging to higher-fitness subsets.

  The implementation treated a mismatch between the published reinforcement equation and its explanation as a likely typo. Reinforcement was made proportional to the non-negative fitness score so that better-performing subsets received stronger reinforcement. This was an implementation adaptation rather than an exact reproduction of the published equation.

- **Particle Swarm Optimisation (PSO):** PSO was implemented using a custom `BinaryPSO` class, adapted from the binary feature-selection method described by Tong and Wu[19]. Each particle represented a possible feature subset, with a value of 1 indicating that a feature was selected and 0 indicating that it was excluded.

  During the search, each particle updated its velocity using its own best solution found so far and the best solution found by the swarm. A sigmoid function converted the velocity values into probabilities, which were then used to decide whether each feature would be selected in the next solution.

  In this study, the inertia weight was set to 0.9, the cognitive coefficient to 0.5, and the social coefficient to 0.3. Velocity values were limited to the range $[-6, 6]$. Candidate subsets were evaluated using the same Random Forest-based fitness function as the other optimisers.

- **Enhanced RLGWO:** Enhanced RLGWO was used through the imported `EnhancedBinaryRLGWO` implementation, following Sammour et al.'s feature-selection framework [2]. The underlying Grey Wolf Optimiser updates candidates using the three highest-fitness population members. Binary masks indicate which features are selected.

  The source framework uses a Dueling Deep Q-Network to choose search-control actions, Prioritised Experience Replay to support network training, and Opposition-Based Learning to generate alternative candidates. These mechanisms are intended to balance exploration and exploitation; their presence does not establish that useful policy learning occurred in the present run.

  The implementation used the same candidate pool, fitness and feature-count constraints as ACO and PSO. Internal DQN updates in the imported module were not audited by the corrected notebook. The reported results therefore concern the tested implementation as a whole and do not isolate the contribution of learned control. This study applied and evaluated an existing framework rather than inventing Enhanced RLGWO.

4.4.2 Baseline implementation phase

The overall experimental pipeline consisted of three phases. In the baseline implementation phase, the ISP-derived DNS exfiltration dataset provided by Žiža et al. was loaded, retaining the 17 engineered DNS features and their associated class labels. Stratified sampling was used to divide the data into training, validation, and test partitions using a 70/10/20 split.

Random Forest and XGBoost were trained using all 17 features to establish the full-feature baselines. Their configurations were informed by the settings reported by Žiža et al., with the implementation choices and differences documented in the classifier configuration subsections.

The trained baseline models were evaluated on the test partition using attack-class precision, recall, and F1-score. These results provided the reference against which the reduced-feature models were compared. Comparisons with the original study's reported results were interpreted in light of differences in data allocation and model configuration.

4.4.3 Feature-selection phase

ACO, PSO, and Enhanced RLGWO were used to search for compact subsets of the 17 candidate DNS features. Each candidate solution was represented by a binary vector:

$$\mathbf{m} = (m_1, m_2, \ldots, m_{17}), \qquad m_j \in \{0, 1\}, \tag{1}$$

where $m_j = 1$ indicates that feature j is included and $m_j = 0$ indicates that it is excluded. The number of selected features is therefore:

$$k(\mathbf{m}) = \sum_{j=1}^{17} m_j. \tag{2}$$

To reduce the computational cost of repeated fitness evaluations, the search used a stratified subsample of 500,000 records from the training partition and a stratified sample of 150,000 records from the validation partition. Both samples used seed 42. For each admissible mask, a Random Forest with 50 trees, `random_state=42` and `n_jobs=1` was fitted on the selected training columns. Maximum tree depth was not explicitly bounded. Validation-subsample predictions were used to calculate weighted F1-score and false-positive rate.

All three optimisers used this validation-based fitness function. The test partition was not used to calculate candidate fitness or choose feature subsets in the corrected experiment. The procedure repaired the earlier direct test-sampling overlap while retaining the existing data partitions. Earlier inspection of test results and unverified request-window independence remain separate limitations.

The fitness function was adapted from the F1-based, feature-count-penalised approach described by Sammour et al. [2]. For each candidate subset, a 50-tree Random Forest was trained on the search-training sample and evaluated on the search-validation sample. The optimiser favoured subsets with high weighted F1-score, a low false-positive rate, and a feature count close to nine.

Subsets containing fewer than eight or more than ten features were assigned a fitness score of zero. For admissible subsets, the fitness score was calculated as the weighted F1-score minus the false-positive rate and minus 0.002 multiplied by the absolute difference between the number of selected features and nine. The same fitness rule was used for ACO, PSO, and Enhanced RLGWO.

The false-positive rate is:

$$\mathrm{FPR} = \frac{\mathrm{FP}}{\mathrm{FP} + \mathrm{TN}}, \tag{3}$$

where FP is the number of benign records incorrectly classified as attacks and TN is the number correctly classified as benign. The false-positive rate enters the fitness function as a proportion, not as a percentage.

All three optimisers maximised this fitness score. The first term rewarded predictive performance, the second penalised false alarms, and the third favoured subsets containing nine features. Subsets containing fewer than eight or more than ten features were assigned a fitness of zero.

After the search, the best subset returned by each optimiser was used to train the final Random Forest and XGBoost classifiers on the full training set. Their performance was subsequently evaluated on the full test set.

4.5 Evaluation and statistical analysis

Full-feature and selected-subset models were evaluated on the same test partition. Precision, recall and F1-score were reported for the exfiltration-labelled class, alongside false-positive rate and confusion-matrix counts. Weighted F1 used within search fitness was distinguished from attack-class F1 used for final reporting.

Training time covered the classifier fit operation, including validation monitoring for XGBoost. Batch inference time covered prediction on the complete test par-

tition. Feature-column slicing, feature extraction, aggregate-window formation and metaheuristic search were excluded from classifier timings.Table 8 reports the timings from the seed-42 evaluation run. Timing variation was assessed separately in the three-seed repeated-run analysis reported in Section 4.6 and Table 9.

Pairwise exact two-sided McNemar tests compared correctness of the four feature configurations within each classifier. Holm adjustment was applied across twelve comparisons at an adjusted significance threshold of 0.05. These tests concern paired correctness, not direct tests of attack-class F1 or variability across independent optimiser runs. Their interpretation also depends on independent evaluation units, which were not established for related DNS windows.

The existing test partition had been inspected during earlier development. Although excluded from corrected search fitness, it was not a newly reserved, previously uninspected holdout.

4.6 Repeated-run robustness analysis

To assess the repeatability of the final model evaluation, each baseline and reduced-feature configuration was retrained and evaluated using three random seeds (42, 43, and 44). The training, validation, and test partitions, classifier hyperparameters, and selected feature subsets were held fixed across repetitions. For every configuration, attack-class precision, recall, F1-score, false-positive count, false-negative count, training time, and median batch-inference time were recorded. Mean and standard deviation were then calculated across the three runs.

This analysis assesses the stability of final classifier evaluation and recorded computational measurements for the fixed selected subsets. It does not test whether the metaheuristic searches themselves produce identical feature subsets under different optimiser seeds, because the selected subsets were held constant during the repeated evaluation.

4.7 Tools and implementation

The experiment was implemented in Python using NumPy and pandas for data handling, scikit-learn for Random Forest, stratified sampling and classification metrics, and the XGBoost package for `XGBClassifier`. SciPy supplied the exact binomial tests used in McNemar comparisons. Matplotlib was used for result visualisation.

ACO and binary PSO were implemented as custom classes. `EnhancedBinaryRLGWO` was imported from the local `Enhanced_detection.py` module based on the framework attributed to Sammour et al. [2]. The reported searches did not use jostar or Py-FS.

The corrected notebook loaded six cached numerical arrays representing training, validation and test features and labels. It saved new search records, selected indices, predictions, confusion matrices, evaluation summaries and classifier

parameters. Final timings were measured on the same fits used for detection evaluation, not an earlier classifier configuration.

The main seed was 42. Python and NumPy seeds were reset for each search and the PyTorch seed was set when available. This documents seed handling rather than guaranteeing identical results across software versions or hardware. Reproducibility requires the matching cached data, local optimiser module and execution environment.

5 Results and Discussion

5.1 Baseline performance with all 17 features

Random Forest and XGBoost were evaluated using all 17 engineered DNS features to establish full-feature baselines. The test set contained 6,979,874 benign records and 43,416 attack records. Given this class imbalance, attack-class precision, recall, and F1-score were examined alongside the false-positive rate, rather than relying on overall accuracy alone.

The full-feature XGBoost model achieved an attack-class precision of 0.997327, recall of 0.996706, and F1-score of 0.997016. Random Forest achieved a precision of 0.999608, recall of 0.998457, and F1-score of 0.999032. Random Forest therefore produced higher values across all three attack-class metrics in the baseline evaluation.

The confusion-matrix counts provide a more concrete interpretation of this difference. Random Forest produced 17 false positives and 67 false negatives, whereas XGBoost produced 116 false positives and 143 false negatives. Their false-positive rates were approximately 0.000244% and 0.001662%, respectively.

These results establish strong reference points for evaluating feature selection. However, the observed performance difference does not by itself identify its cause. Although Random Forest and XGBoost use different ensemble-learning mechanisms, the experiment did not isolate the effects of ensemble structure, hyperparameter settings, or class imbalance. The difference should therefore be interpreted as a result of the evaluated configurations rather than evidence that one classifier is inherently more robust to imbalance.

5.2 Selected feature subsets

ACO, PSO, and Enhanced RLGWO each selected nine of the seventeen candidate features, corresponding to a feature-count reduction of approximately 47.06%. The selected indices are reported in Table 4, with feature names corresponding to the index mapping in Table 5.

Although the optimisers returned subsets of the same size, this outcome should be interpreted in the context of the search constraints. Subsets were restricted to between eight and ten features, and the fitness function included a penalty for deviation from nine features. The common subset size therefore does not

establish that nine is the optimal number of features outside the specified search setting.

Table 4: Feature subsets selected by the three optimisers.

Optimiser	Selected indices	Number	Reduction (%)
ACO	0, 2, 4, 6, 7, 9, 10, 12, 13	9	47.06
PSO	0, 1, 3, 6, 7, 8, 12, 14, 15	9	47.06
Enhanced RLGWO	0, 1, 2, 7, 10, 12, 13, 14, 15	9	47.06

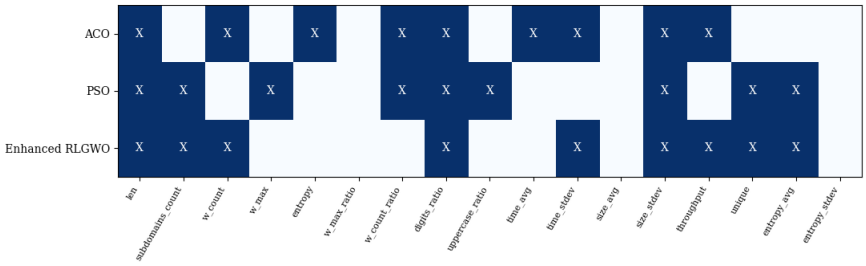


Figure 2: Feature-selection matrix for ACO, PSO and Enhanced RLGWO. Marked cells indicate selected features. Each returned subset contains nine features under the shared search constraints.

Index	Feature Name
0	len
1	subdomains_count
2	w_count
3	w_max
4	entropy
5	w_max_ratio
6	w_count_ratio
7	digits_ratio
8	uppercase_ratio
9	time_avg
10	time_stdev
11	size_avg
12	size_stdev
13	throughput
14	unique
15	entropy_avg
16	entropy_stdev

Table 5: The 17 candidate DNS traffic features and their index positions.

Three features were included in every selected subset: `len`, `digits_ratio`, and `size_stdev`. Their repeated selection suggests that they were useful components of the subsets found under the shared fitness function. However, this agreement does not mean that these three features alone are sufficient for detection or that they would be selected consistently across different random seeds.

Enhanced RLGWO shared six features with ACO: `len`, `w_count`, `digits_ratio`, `time_stdev`, `size_stdev`, and `throughput`. It also shared six features with PSO: `len`, `subdomains_count`, `digits_ratio`, `size_stdev`, `unique`, and `entropy_avg`. ACO and PSO shared four features: `len`, `w_count_ratio`, `digits_ratio`, and `size_stdev`.

Enhanced RLGWO therefore combined features that also appeared in the other two subsets, while selecting a different overall combination. Relative to ACO, its subset included `subdomains_count`, `unique`, and `entropy_avg` instead of `entropy`, `w_count_ratio`, and `time_avg`. Relative to PSO, it included `w_count`, `time_stdev`, and `throughput` instead of `w_max`, `w_count_ratio`, and `uppercase_ratio`.

Because all three subsets contained nine features, their performance differences cannot be explained by feature count alone. The results instead compare different feature combinations under a common dimensionality constraint. Nevertheless, attributing an improvement to any individual feature would require additional experiments.

5.3 Detection performance of the selected feature subsets

Table 6 compares the full-feature baselines with the optimiser-selected subsets. Precision, recall, and F1-score refer to the attack class. False-positive rate is expressed as a percentage of benign records incorrectly classified as attacks.

Table 6: Test-set detection performance. Precision, recall, and F1-score refer to the attack class; FPR is expressed as a percentage. RF denotes Random Forest.

Configuration	Classifier	Precision	Recall	F1	FPR (%)
Baseline	XGBoost	0.997327	0.996706	0.997016	0.001662
Baseline	RF	0.999608	0.998457	0.999032	0.000244
ACO	XGBoost	0.995782	0.995186	0.995484	0.002622
ACO	RF	0.999075	0.995278	0.997173	0.000573
PSO	XGBoost	0.998545	0.995808	0.997175	0.000903
PSO	RF	0.999422	0.996430	0.997924	0.000358
Enhanced RLGWO	XGBoost	0.998294	0.997236	0.997765	0.001060
Enhanced RLGWO	RF	0.999377	0.997282	0.998328	0.000387

Enhanced RLGWO achieved the highest attack-class F1-score among the three feature-selection methods for both classifiers. With XGBoost, Enhanced RLGWO achieved 0.997765, followed by PSO at 0.997175 and ACO at 0.995484. Enhanced RLGWO also exceeded the full-feature XGBoost baseline, which achieved 0.997016.

However, Enhanced RLGWO did not achieve the best value for every individual metric. PSO produced the highest XGBoost precision of 0.998545 and the lowest false-positive rate of 0.000903%. Enhanced RLGWO achieved higher recall of 0.997236, compared with 0.995808 for PSO. Its higher F1-score therefore reflects a different balance between false alarms and missed attack records.

With Random Forest, the full-feature baseline remained the strongest configuration, achieving an F1-score of 0.999032. Enhanced RLGWO achieved the highest F1-score among the selected subsets at 0.998328, followed by PSO at 0.997924 and ACO at 0.997173. All three selected subsets therefore had lower Random Forest F1-scores than the full-feature baseline.

The ranking also depended on the classifier. Although Random Forest achieved a higher F1-score than XGBoost for the baseline, ACO, and PSO configurations, XGBoost achieved the higher F1-score with the Enhanced RLGWO-selected subset. This demonstrates that the performance of a selected feature combination cannot be assessed independently of the classifier used to evaluate it.

Figure 3 summarises these differences. The high scores across configurations should not obscure the remaining errors. Small absolute changes in F1-score can correspond to different numbers of false alarms and missed attack records, as examined below.

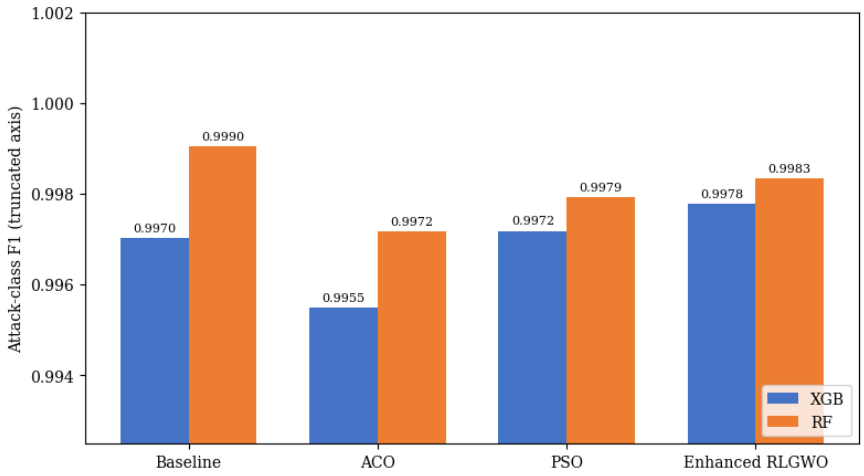


Figure 3: Attack-class F1-scores for the full-feature baselines and optimiser-selected subsets.

5.4 Error analysis

Table 7 reports the confusion-matrix counts. True negatives (TN) and true positives (TP) signify correctly classified benign and attack records, respectively. False positives (FP) are benign records classified as attacks, while false negatives (FN) are attack records classified as benign.

Table 7: Test-set confusion-matrix counts. Counts refer to records, not distinct attack incidents.

Configuration	Classifier	TN	FP	FN	TP
Baseline	XGBoost	6,979,758	116	143	43,273
Baseline	RF	6,979,857	17	67	43,349
ACO	XGBoost	6,979,691	183	209	43,207
ACO	RF	6,979,834	40	205	43,211
PSO	XGBoost	6,979,811	63	182	43,234
PSO	RF	6,979,849	25	155	43,261
Enhanced RLGWO	XGBoost	6,979,800	74	120	43,296
Enhanced RLGWO	RF	6,979,847	27	118	43,298

For XGBoost, Enhanced RLGWO reduced false positives from 116 to 74 and false negatives from 143 to 120 relative to baseline. Total errors decreased from 259 to 194, corresponding to a reduction of approximately 25.10%. This improvement involved fewer errors of both types, rather than exchanging fewer missed attacks for more false alarms.

Compared with PSO, Enhanced RLGWO with XGBoost detected 62 additional attack records but generated 11 additional false positives. With Random Forest, Enhanced RLGWO detected 37 additional attack records compared with PSO while generating two additional false positives.

Enhanced RLGWO also produced fewer errors than ACO for both classifiers. With XGBoost, it produced 109 fewer false positives and 89 fewer false negatives. With Random Forest, it produced 13 fewer false positives and 87 fewer false negatives.

Nevertheless, the full-feature Random Forest remained the configuration with the fewest total errors. It produced 84 errors, compared with 145 for Enhanced RLGWO with Random Forest. Relative to this baseline, the Enhanced RLGWO subset introduced ten additional false positives and 51 additional false negatives.

Figure 4 illustrates these error patterns. The distinction between false positives and false negatives is important because configurations with similar F1-scores may impose different operational costs. The preferred configuration therefore depends partly on the relative costs of missed attacks and false alarms.

5.5 Training and inference time

Table 8 reports classifier training and batch inference times from the seed-42 evaluation run. The repeated-run mean and standard deviation results are reported separately in Table 9. Percentage reductions are calculated relative to the baseline of the same classifier:

$$R_t = 100 \times \left(\frac{t_{\text{baseline}} - t_{\text{subset}}}{t_{\text{baseline}}} \right). \tag{4}$$

Positive values indicate shorter execution times; zero indicates no change, while negative values indicate longer execution times.

Table 8: Training and batch-prediction times from the seed-42 evaluation run. Percentage reductions are measured against the baseline for the same classifier. Negative values indicate longer recorded times.

Configuration	Model	Training (s)	Reduction (%)	Inference (s)	Reduction (%)
Baseline	XGB	188.331	—	4.863	—
Baseline	RF	481.248	—	2.966	—
ACO	XGB	170.961	9.22	4.740	2.52
ACO	RF	468.837	2.58	3.341	-12.62
PSO	XGB	166.491	11.60	4.097	15.74
PSO	RF	197.172	59.03	3.002	-1.21
Enhanced RLGWO	XGB	163.177	13.36	4.526	6.93
Enhanced RLGWO	RF	427.961	11.07	3.123	-5.26

XGBoost had shorter training times than Random Forest for all four configu-

rations. Enhanced RLGWO produced the shortest observed XGBoost training time of 163.177 seconds, compared with 188.331 seconds for the full-feature baseline, a reduction of 13.36%. PSO and ACO reduced XGBoost training time by 11.60% and 9.22%, respectively.

For Random Forest, PSO produced the largest training-time reduction. Training time decreased from 481.248 seconds to 197.172 seconds, corresponding to a reduction of 59.03%. Enhanced RLGWO reduced training time by 11.07%, while ACO achieved a reduction of 2.58%.

Inference results differed from training results. All three selected subsets reduced XGBoost batch inference time. PSO achieved the largest reduction of 15.74%, followed by Enhanced RLGWO at 6.93% and ACO at 2.52%.

None of the selected subsets reduced Random Forest inference time relative to baseline. The baseline required 2.966 seconds, compared with 3.002 seconds for PSO, 3.123 seconds for Enhanced RLGWO, and 3.341 seconds for ACO. The difference between baseline and PSO was only approximately 0.036 seconds.

Figure 5 shows that shorter training times did not necessarily translate into shorter inference times. These measurements do not identify a particular feature as the cause of a timing difference. Model structure and execution variability may also affect the measurements.

The reported values represent single recorded runs. They measure classifier fitting and batch prediction, not the complete detection pipeline. Feature extraction, aggregate-window formation, and metaheuristic search costs are not included. Consequently, these results should not be interpreted as measurements of end-to-end detection latency.

5.6 Repeated-run robustness analysis

To evaluate repeatability, each configuration was retrained and evaluated with three random seeds (42, 43, and 44). Attack F1-score variation was low across the repeated runs. For Random Forest, the full 17-feature baseline achieved the highest mean attack-class F1-score (0.99887 ± 0.00020). Among the reduced subsets, Enhanced RLGWO achieved the highest mean F1-score (0.99835 ± 0.00004), followed by PSO (0.99800 ± 0.00009) and ACO (0.99709 ± 0.00031).

PSO provided the strongest Random Forest training-efficiency trade-off. Its 9-feature subset reduced mean training time from 500.93 ± 68.74 seconds for the baseline to 214.35 ± 48.45 seconds, a 57.2% reduction, while decreasing mean attack F1-score by only 0.00086. Enhanced RLGWO achieved the fastest mean Random Forest inference time, 3.236 ± 0.050 seconds, compared with 3.410 ± 0.328 seconds for the baseline.

For XGBoost, Enhanced RLGWO produced the highest mean attack-class F1-score, 0.99786 ± 0.00008, exceeding the full-feature baseline mean of 0.99713 ± 0.00058. This advantage was observed in all three repeated runs. Therefore, the repeated-run evaluation supports the stability of the Enhanced RLGWO–XGBoost result. In contrast, the relative ordering of PSO and the XGBoost baseline was not consistent across every seed.

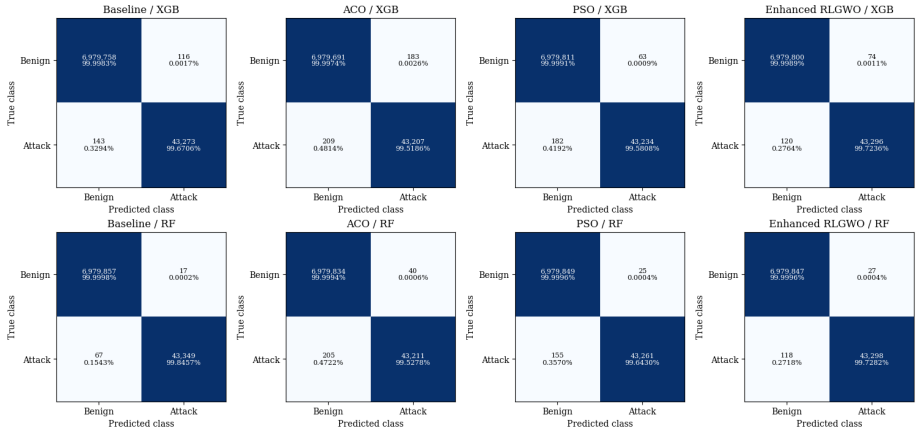


Figure 4: Confusion matrices for XGBoost (top row) and Random Forest (bottom row). Rows represent true classes and columns represent predicted classes. Each cell reports its count and percentage within the true class.

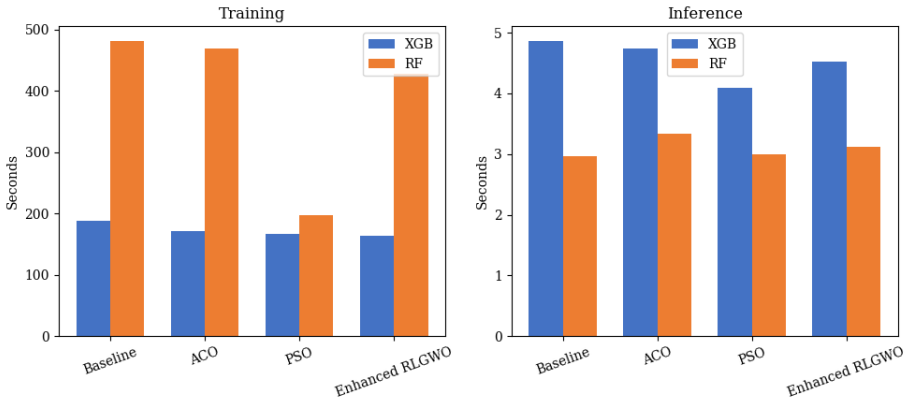


Figure 5: Recorded classifier training times (left) and batch inference times (right). Measurements exclude feature extraction and metaheuristic search time.

Because feature subsets were held fixed, these findings show repeatability of model evaluation and measured computation costs, rather than stability of the metaheuristic search outcome itself.

Table 9: Repeated-run results across three random seeds. Values are mean ± standard deviation.

Classifier	Configuration	Attack F1	Training time (s)	Inference time (s)
RF	Baseline	0.99887 ± 0.00020	500.93 ± 68.74	3.410 ± 0.328
RF	ACO	0.99709 ± 0.00031	491.45 ± 72.60	3.394 ± 0.094
RF	PSO	0.99800 ± 0.00009	214.35 ± 48.45	3.394 ± 0.656
RF	Enhanced RLGWO	0.99835 ± 0.00004	402.60 ± 7.62	3.236 ± 0.050
XGB	Baseline	0.99713 ± 0.00058	190.61 ± 41.30	4.158 ± 1.152
XGB	ACO	0.99551 ± 0.00008	166.81 ± 5.83	4.701 ± 0.532
XGB	PSO	0.99604 ± 0.00101	170.31 ± 12.97	4.553 ± 0.386
XGB	Enhanced RLGWO	0.99786 ± 0.00008	168.80 ± 13.53	3.916 ± 0.147

5.7 Balancing Detection Performance and Runtime

Enhanced RLGWO with XGBoost combined improved attack-class detection metrics with shorter recorded training and inference times relative to the full-feature XGBoost baseline. Its F1-score increased from 0.997016 to 0.997765, while total errors decreased from 259 to 194. Training and batch inference times decreased by 13.36% and 6.93%, respectively.

However, Enhanced RLGWO did not dominate every alternative. PSO with XGBoost achieved higher precision, a lower false-positive rate, and shorter inference time, but lower recall and F1-score. Similarly, PSO with Random Forest trained substantially faster than Enhanced RLGWO with Random Forest, but produced more false negatives.

The full-feature Random Forest remained the strongest detector overall according to attack-class F1-score and total error count. Its training time was longer than those of the selected-subset Random Forest models, but its recorded inference time was the shortest among them.

The findings therefore support different configurations for different priorities. Enhanced RLGWO with XGBoost provided a favourable observed balance between detection performance and classifier runtime relative to its own baseline. PSO with Random Forest provided the largest training-time saving, whereas the full-feature Random Forest minimised observed classification errors.

5.8 Statistical comparison

McNemar's tests were used to compare pairs of feature configurations evaluated on the same test records [20]. The tests focused on records where one configura-

tion made a correct prediction and the other made an incorrect prediction. For each comparison:

- n_{01} is the number of records that Configuration A classified correctly but Configuration B classified incorrectly.
- n_{10} is the number of records that Configuration A classified incorrectly but Configuration B classified correctly.

Exact two-sided p-values were calculated using these two counts. Because twelve comparisons were performed, Holm's method was used to adjust the p-values and control the risk of falsely declaring a difference statistically significant across the comparisons [21]. A difference was considered statistically significant when its adjusted p-value was below 0.05. The results are reported in Table 10.

Table 10: Pairwise McNemar tests on test-set classification correctness. Holm-adjusted p-values account for all twelve comparisons. RF denotes Random Forest and ERLGWO denotes Enhanced RLGWO.

Classifier	Config. A	Config. B	n_{01}	n_{10}	Exact p	Holm-adjusted p
XGBoost	Baseline	ACO	296	163	5.50930×10^{-10}	3.85651×10^{-9}
XGBoost	Baseline	PSO	157	171	0.472933	0.472933
XGBoost	Baseline	ERLGWO	110	175	0.000141157	0.000564629
XGBoost	ACO	PSO	144	291	1.57963×10^{-12}	1.26371×10^{-11}
XGBoost	ACO	ERLGWO	98	296	3.29407×10^{-24}	3.62348×10^{-23}
XGBoost	PSO	ERLGWO	108	159	0.00215469	0.00646406
RF	Baseline	ACO	178	17	5.13950×10^{-35}	6.16740×10^{-34}
RF	Baseline	PSO	122	26	4.54768×10^{-16}	4.54768×10^{-15}
RF	Baseline	ERLGWO	82	21	1.06968×10^{-9}	6.41805×10^{-9}
RF	ACO	PSO	95	160	0.0000562469	0.000281234
RF	ACO	ERLGWO	51	151	1.11343×10^{-12}	1.00209×10^{-11}
RF	PSO	ERLGWO	77	112	0.0131834	0.0263667

For XGBoost, Enhanced RLGWO produced significantly better paired classification correctness than the full-feature baseline ($p_{\text{Holm}} = 0.000565$). In this comparison, Enhanced RLGWO correctly classified 175 records that the baseline misclassified, whereas the baseline correctly classified 110 records that Enhanced RLGWO misclassified. This corresponds to 65 fewer total errors for Enhanced RLGWO, consistent with the confusion-matrix results.

Enhanced RLGWO also significantly outperformed PSO ($p_{\text{Holm}} = 0.006464$) and ACO ($p_{\text{Holm}} = 3.62348 \times 10^{-23}$) when evaluated using XGBoost. PSO significantly outperformed ACO, but its comparison with the full-feature baseline did not reach significance ($p_{\text{Holm}} = 0.472933$). This non-significant result does not establish equivalence between PSO and the baseline.

For Random Forest, the full-feature baseline significantly outperformed all three reduced-feature configurations. Its advantage over Enhanced RLGWO remained significant after Holm adjustment ($p_{\text{Holm}} = 6.41805 \times 10^{-9}$). Consequently, the reduction in feature count did not preserve baseline Random Forest correctness without a statistically detectable loss.

Among the reduced-feature Random Forest configurations, Enhanced RLGWO significantly outperformed both ACO ($p_{\text{Holm}} = 1.00209 \times 10^{-11}$) and PSO ($p_{\text{Holm}} = 0.026367$). PSO also significantly outperformed ACO.

Taken together, the paired tests support Enhanced RLGWO's advantage over the other two feature selectors for both classifiers in this experiment. However, its relationship with the full-feature baseline was classifier-dependent: it improved XGBoost correctness but reduced Random Forest correctness. McNemar's test concerns paired classification errors and does not directly establish the statistical significance of differences in attack-class precision, recall, or F1-score.

5.9 Limitations of the evaluation

5.10 Summary

Enhanced RLGWO selected nine of the seventeen features, reducing the number of features by 47.06%. It achieved the highest attack-class F1-score of the three feature-selection methods for both classifiers in the reported evaluation. With XGBoost, its attack-class F1-score increased from 0.997016 to 0.997765, while total classification errors decreased from 259 to 194. Recorded training time and batch inference time decreased by 13.36% and 6.93%, respectively.

Exact McNemar tests with Holm adjustment showed that Enhanced RLGWO had significantly better paired classification correctness than ACO and PSO when comparing paired correct and incorrect predictions for both classifiers. Its improvement over the full-feature XGBoost baseline was also statistically significant ($p_{\text{Holm}} = 0.000565$). However, these tests compare paired prediction correctness and do not directly compare attack-class F1-scores. They should also be interpreted carefully because DNS records may not be fully independent.

The results depended on the classifier used. The full-feature Random Forest achieved the highest attack-class F1-score overall, at 0.999032, and performed significantly better than all three reduced-feature Random Forest models. PSO gave the largest reduction in Random Forest training time, at 59.03%. However, none of the selected feature subsets reduced the recorded Random Forest batch inference time compared with the full-feature model.

These results show a trade off between detection performance and classifier execution time. Enhanced RLGWO with XGBoost improved attack-class F1-score and reduced recorded training and batch inference time compared with the full-feature XGBoost baseline. However, the results apply only to the data, feature subsets, classifier settings, and search settings used in this study. The repeated tests showed consistent classifier results when the same selected features were used, but they did not repeat the feature selection searches with different

optimiser seeds. Further tests using new datasets, independent holdout data, different search settings, and repeated optimiser runs are needed before claiming that one optimiser is generally better than the others.

6 Conclusion and Future Work

6.1 Limitations of the Evaluation

The results only apply to the feature subsets, classifier settings, and tests used in this study. They do not prove that one optimiser will always work better than the others when different random seeds, datasets, or search settings are used.

The search score used weighted F1-score and a false-positive penalty. Weighted F1 gives more importance to the larger benign class. Therefore, the search did not directly try to get the highest attack-class F1-score. The search only tested feature subsets with eight to ten features. It did not test whether fewer features could still give good results.

Random Forest was used to score feature subsets during the search. This means that the selected features may work best with Random Forest. Testing the same features with XGBoost shows whether they also work with another classifier. However, it does not show that an XGBoost-based search would choose the same features.

The test results include all attack records together. They do not show separate results for each tunnelling tool, changed exfiltration setting, or completely new attack type. The test data were also looked at during earlier development. Although they were not used for feature selection in the corrected experiment, they were not a fully new and unseen test set.

The McNemar tests checked whether two models made correct or incorrect predictions on the same test records. They did not directly compare attack-class F1-scores or test differences between repeated optimiser runs. These tests also assume that the test records are independent. However, DNS records may be related through the same domain, host, or traffic sequence. Therefore, the p-values should be interpreted with care.

The study did not compare the metaheuristic methods with simple methods, such as random feature selection or selecting features by importance. Therefore, it cannot show whether the metaheuristic methods are better than simpler methods with the same search budget. The reinforcement-learning part of the imported Enhanced RLGWO code was also not checked in detail. Therefore, the results cannot be linked only to the reinforcement-learning part of the method.

The final classifiers were tested three times using different random seeds, while keeping the selected features the same. However, the optimiser searches were not run again using different optimiser seeds. The timing results measured only classifier training and batch prediction time. They did not include feature extraction, aggregate-window creation, optimiser search time, or the total time needed in a real deployment.

References

[1] K. Žiža and P. T. P. Vuletić, "Dns exfiltration detection in the presence of adversarial attacks and modified exfiltrator behaviour," *International Journal of Information Security*, vol. 22, pp. 1865–1880, 07 2023.

[2] M. Sammour, M. F. I. Othman, A. Hassan, O. Bhais, and M. S. Talib, "Advanced dns tunneling detection: a hybrid reinforcement learning and metaheuristic approach," *Frontiers in Computer Science*, vol. 7, 01 2026.

[3] Z. Sui, H. Shu, F. Kang, Y. Huang, and G. Huo, "A comprehensive review of tunnel detection on multilayer protocols: From traditional to machine learning approaches," *Applied Sciences*, vol. 13, p. 1974, 01 2023. [Online]. Available: https://www.mdpi.com/2076-3417/13/3/1974

[4] O. Abualghanam, H. Alazzam, B. Elshqeirat, M. Qatawneh, and M. A. Almaiah, "Real-time detection system for data exfiltration over dns tunneling using machine learning," *Electronics*, vol. 12, pp. 1467–1467, 03 2023.

[5] C. Zhang, D. Jia, L. Wang, W. Wang, F. Liu, and A. Yang, "Comparative research on network intrusion detection methods based on machine learning," *Computers Security*, vol. 121, p. 102861, 10 2022.

[6] K. Žiža, P. Vuletić, and P. Tadić, "Dns exfiltration dataset," 2022. [Online]. Available: https://doi.org/10.17632/c4n7fckkz3.2

[7] M. Aiello, M. Mongelli, and G. Papaleo, "Dns tunneling detection through statistical fingerprints of protocol messages and machine learning," *International Journal of Communication Systems*, vol. 28, pp. 1987–2002, 07 2014.

[8] K. Wu, Y. Zhang, and T. Yin, "Ftpb: A three-stage dns tunnel detection method based on character feature extraction," *2021 IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, pp. 250–258, 12 2020.

[9] M. Mazini, B. Shirazi, and I. Mahdavi, "Anomaly network-based intrusion detection system using a reliable hybrid artificial bee colony and adaboost algorithms," *Journal of King Saud University – Computer and Information Sciences*, vol. 31, no. 4, pp. 541–553, 2019.

[10] H. S. Talabani, Z. K. Abdul, and H. M. Mohammed Saleh, "Dns over https tunneling detection system based on selected features via ant colony optimization," *Future Internet*, vol. 17, p. 211, 05 2025.

[11] G. Alshammari, A. H. Lashkari, and A. A. Ghorbani, "Cira–cic–dohbrw–2020: A dataset for dns over https traffic analysis," 2020, canadian Institute for Cybersecurity, University of New Brunswick.

[12] R. Mitsuhashi, Y. Jin, K. Iida, T. Shinagawa, and Y. Takai, "Malicious dns tunnel tool recognition using persistent doh traffic analysis," *IEEE Transactions on Network and Service Management*, pp. 1–1, 2022.

[13] K. Jerabek, K. Hynek, O. Rysavy, and I. Burgetova, "Dns over https detection using standard flow telemetry," *IEEE Access*, vol. 11, pp. 50 000–50 012, 01 2023. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10123708/

[14] N. Sharma, M. Swarnkar, and Divyanshu, "Dlaze: Detecting dns tunnels using lightweight and accurate method for zero-day exploits," *IEEE Transactions on Network and Service Management*, pp. 1–1, 2025.

[15] J. Li, H. Zhang, Y. Liu, and Z. Liu, "Semi-supervised machine learning framework for network intrusion detection," *The Journal of supercomputing/Journal of supercomputing*, vol. 78, pp. 13 122–13 144, 03 2022.

[16] A. A. Awad, A. F. Ali, and T. Gaber, "An improved long short term memory network for intrusion detection," *PLOS ONE*, vol. 18, pp. e0 284 795–e0 284 795, 08 2023.

[17] S. Wang, L. Sun, S. Qin, W. Li, and W. Liu, "Krtunnel: Dns channel detector for mobile devices," *Computers Security*, vol. 120, pp. 102 818–102 818, 06 2022.

[18] J. Liang, S. Wang, S. Zhao, and S. Chen, "Fecc: Dns tunnel detection model based on cnn and clustering," *Computers Security*, vol. 128, pp. 103 132–103 132, 05 2023.

[19] L. Tong and Q. Wu, "Intrusion feature selection algorithm based on particle swarm optimization," *International Journal of Computer Science and Network Security*, vol. 14, no. 12, 2014. [Online]. Available: http://paper.ijcsns.org/07_book/201412/20141208.pdf

[20] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[21] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979. [Online]. Available: https://www.jstor.org/stable/4615733